

UN EXERCICE PAR COPIE

Une annexe est disponible à la fin du sujet et contient la description des méthodes dont vous pourriez avoir besoin.

Inutile de fournir les instructions d'import. Sauf explicitement demandé, vous pouvez supposer que tous les accesseurs dont vous avez besoin sont disponibles : pas besoin de les écrire.

Vous êtes libre de décomposer votre code et d'ajouter des méthodes dans ce but. À vous de penser à la surcharge ou au chaînage ! Par ailleurs, si pour répondre à une question, vous avez besoin d'écrire du code dans différentes classes, n'oubliez pas de le spécifier clairement avant votre code.

Exercice 1 : Bientôt les vacances !

Dans cet exercice, nous allons modéliser les vols en avion pour préparer les excursions d'une agence de voyages. Nous nous intéresserons en particulier aux horaires, aux aéroports, aux vols et bien sûr, aux vacances !

Q1. Occupons-nous d'abord des horaires.

Q1.1. Écrivez la classe `Horaire` qui est caractérisée par une date, une heure (une valeur comprise entre 0 et 23, bornes incluses) et un nombre de minutes (une valeur comprise entre 0 et 59, bornes incluses). Cette classe bénéficie de deux constructeurs : l'un complètement spécifié, l'autre qui repose par défaut sur la date du jour quand elle n'est pas fournie. Notez qu'on affecte 0 aux attributs de type entiers dont la valeur passée en paramètre est incorrecte.

Horaire
- date : LocalDate
- heure : int
- minute : int
...

Q1.2. Écrivez la méthode `equals` déterminant l'égalité entre deux instances de cette classe `Horaire`.

```
boolean equals(Object o)
```

Q1.3. Écrivez les méthodes `isBefore` qui renvoie `true` si l'heure indiqué dans l'objet courant est avant celui passé en paramètre. Si on ne fournit que les heures et les minutes en paramètre, on ignorera la date. Si on fournit un `Horaire` en paramètre, on prend l'intégralité de l'objet en compte.

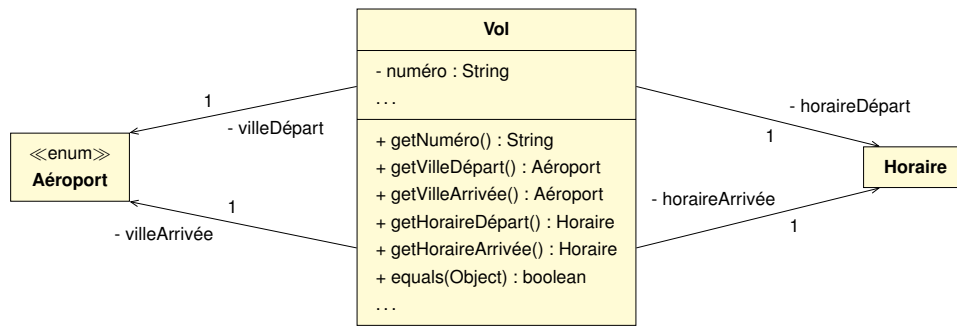
```
boolean isBefore(int heure, int minute)
boolean isBefore(Horaire other)
```

Q2. Concentrons-nous sur les aéroports.

Q2.1. Écrivez l'énumération `Aéroport` sachant que chaque aéroport est caractérisé par un code de 3 lettres, le nom de l'aéroport et la ville dans laquelle il se situe. Pour simplifier, nous ne considérons ici que 4 aéroports. Ne fournissez que le constructeur et l'accesseur pour la ville.

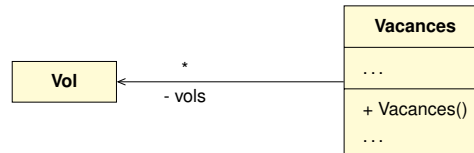
- **CDG**, Charles-De-Gaulle, Paris
- **LHR**, rethrow, Londres
- **MAD**, Barajas, Madrid
- **ORY**, Orly, Paris

Nous ne vous demandons pas d'écrire la classe `Vol` : elle est fournie prête à l'emploi et répond aux spécifications UML suivantes :



Q3. Intéressons-nous enfin aux `Vacances` ! Cette classe va permettre de fournir la liste des vols disponibles.

Q3.1. Sachant qu'une instance est caractérisée par une `List` de `Vol` (qui est vide à la création), écrivez la classe `Vacances` en respectant les spécifications UML suivantes :



Q3.2. Ajoutez une méthode `ajouterVol` qui permet d'ajouter un vol au planning. La méthode ne fait pas l'insertion et renvoie `false` si le vol est déjà présent dans la liste.

```
boolean ajouterVol(Vol unVol)
```

Q3.3. Écrivez une méthode `choixVol` qui fournit une liste de tous les vols arrivant à l'aéroport passé en paramètre (quel que soit l'aéroport de départ).

```
List<Vol> choixVol(Aéroport destination)
```

Q3.4. Munissez votre classe `Vacances` d'une méthode `retirerVol` qui filtre la liste et retire tous les vols dont la date précède l'horaire spécifiée en paramètre.

```
void retirerVol(Horaire limite)
```

Q3.5. On souhaite trier les vols selon leurs horaires de départ via une méthode `triDesVols`. Proposez une solution basée sur l'interface `Comparator`. Veillez à bien préciser la ou les classes concernées par la ou les modifications.

```
void triDesVols()
```

Q3.6. On souhaiterait connaître le nombre maximal de vols partant d'un aéroport. Écrivez la méthode `trafficMax` qui calcule ce nombre. Pour faciliter votre algorithme, on vous demande de créer et remplir une table associative, associant à chaque `Aéroport` le nombre de vols qui en partent.

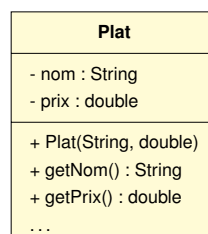
```
int trafficMax()
```

Exercice 2 : À table !

Intéressons-nous dans cet exercice à la gestion des commandes et aux plats servis dans un restaurant. Dans cet exercice, nous vous demanderons de proposer des solutions qui ne sont pas forcément complètement explicitées.

Q1. Commençons par la gestion basique des commandes de plats.

Q1.1. Écrivez la classe `Plat`, sachant qu'un plat est caractérisée par un nom et un prix de base. La classe est dotée d'un constructeur complètement spécifiée et d'un accesseur à chaque attribut.



Q1.2. Une commande est caractérisée par un numéro automatique (`serial` de type `String`) composé de la date du jour suivi d'un numéro de commande. 2024-05-30-42 correspond par exemple à une commande passée le 2024-05-30 avec le numéro 42. Une commande contient également **une table associative** (initialement vide) où l'on associe à chaque plat commandé le nombre d'unité désiré. Écrivez la classe `Commande` en la munissant d'un constructeur sans paramètre réalisant les initialisations nécessaires.

Q1.3. Puisque nous utilisons une table associative, quelles méthodes deviennent nécessaires ? Donnez uniquement la signature de ces méthodes en précisant à quelle classe elles appartiennent.

Q1.4. Ajoutez une méthode `getPrixTotal` qui calcule le prix total d'une commande en fonction des plats et des quantités commandés.

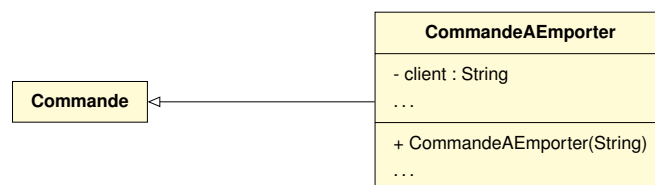
```
double getPrixTotal()
```

Q1.5. Écrivez une méthode `ajoutPlat` qui ajoute un plat dans une quantité donnée à la commande en cours. Si le plat est déjà présent, les quantités sont cumulées.

```
void ajoutPlat(Plat nouveauPlat, int quantité)
```

Q1.6. Rendez la classe `Commande` itérable sur les plats commandés.

Q2. On souhaite gérer des commandes plus spécifiques, notamment les commandes à emporter ou à livrer, puisqu'elles ne sont pas soumises aux mêmes taxes et les clients ne paient que 80% du prix normal. Elles sont également caractérisée par le nom du client (en plus du sérial), pour éviter les erreurs de livraison par exemple. Écrivez la classe `CommandeAEmporter` répondant aux spécifications UML suivantes :



Q3. Le patron souhaite promouvoir les plats végétariens (sur lesquelles il fait honteusement plus de marge de manière indue).

Q3.1. Écrivez une nouvelle classe `PlatVeggie` correspondant aux plats végétariens, et son constructeur.

Q3.2. Ajoutez une méthode `estVeggie` à la classe `Commande` qui détermine si tous les plats commandés sont végétarien. Proposez une décomposition du code en précisant bien les classes modifiées (on évitera les solutions basées sur des `getClass` ou des `instanceof`).

```
boolean estVeggie()
```

Q3.3. Écrivez enfin une méthode `iteratorVeggie` qui fournit un itérateur ne portant que sur les plats végétariens du menu de la commande.

```
Iterator<Plat> iteratorVeggie()
```


Annexe : Méthodes utiles

Classe ArrayList<E>

boolean	<code>add(E e)</code> Appends the specified element to the end of this list.
void	<code>add(int index, E element)</code> Inserts the specified element at the specified position in this list.
boolean	<code>addAll(Collection<E> c)</code> Appends all of the elements in the specified collection to the end of this list, in the order that they are returned by the specified collection's Iterator.
boolean	<code>contains(Object o)</code> Returns true if this list contains the specified element.
E	<code>get(int index)</code> Returns the element at the specified position in this list.
int	<code>indexOf(Object o)</code> Returns the index of the first occurrence of the specified element in this list, or -1 if this list does not contain the element.
boolean	<code>isEmpty()</code> Returns true if this list contains no elements.
int	<code>lastIndexOf(Object o)</code> Returns the index of the last occurrence of the specified element in this list, or -1 if this list does not contain the element.
E	<code>remove(int index)</code> Removes the element at the specified position in this list.
boolean	<code>remove(Object o)</code> Removes the first occurrence of the specified element from this list, if it is present.
boolean	<code>removeAll(Collection<E> c)</code> Removes from this list all of its elements that are contained in the specified collection.
E	<code>set(int index, E element)</code> Replaces the element at the specified position in this list with the specified element.
int	<code>size()</code> Returns the number of elements in this list.

Classe Arrays

<code>static List<T></code>	<code>asList(T... a)</code> Returns a fixed-size list backed by the specified array.
<code>static T[]</code>	<code>copyOf(T[] original, int newLength)</code> Copies the specified array, truncating or padding with nulls (if necessary) so the copy has the specified length.
<code>static T[]</code>	<code>copyOfRange(T[] original, int from, int to)</code> Copies the specified range of the specified array into a new array.
<code>static void</code>	<code>sort(Object[] a)</code> Sorts the specified array of objects into ascending order, according to the natural ordering of its elements.
<code>static String</code>	<code>toString(Object[] a)</code> Returns a string representation of the contents of the specified array.

Classe Collections

static void	sort(List<T> list) Sorts the specified list into ascending order, according to the natural ordering of its elements.
static void	sort(List<T> list, Comparator<T> c) Sorts the specified list according to the order induced by the specified comparator.
static int	frequency(Collection<?> c, Object o) Returns the number of elements in the specified collection equal to the specified object.
static T	max(Collection<? extends T> coll) Returns the maximum element of the given collection, according to the natural ordering of its elements.
static T	max(Collection<T> coll, Comparator<T> comp) Returns the maximum element of the given collection, according to the order induced by the specified comparator.
static T	min(Collection<T> coll) Returns the minimum element of the given collection, according to the natural ordering of its elements.
static T	min(Collection<T> coll, Comparator<T> comp) Returns the minimum element of the given collection, according to the order induced by the specified comparator.

Classe HashMap<K, V>

boolean	containsKey(Object key) Returns true if this map contains a mapping for the specified key.
V	get(K key) Returns the value to which the specified key is mapped, or null if this map contains no mapping for the key.
boolean	isEmpty() Returns true if this map contains no key-value mappings.
V	put(K key, V value) Associates the specified value with the specified key in this map
V	remove(Object key) Removes the mapping for the specified key from this map if present.
int	size() Returns the number of key-value mappings in this map.
Set<K>	keySet() Returns a Set view of the keys contained in this map.
Set<Map.Entry<K, V>>	entrySet() Returns a Set view of the mappings contained in this map.
Collection<V>	values() Returns a Collection view of the values contained in this map.

enum E

<code>int</code>	<code>compareTo(E o)</code> Compares this enum with the specified object for order.
<code>boolean</code>	<code>equals(Object other)</code> Returns true if the specified object is equal to this enum constant.
<code>String</code>	<code>name()</code> Returns the name of this enum constant, exactly as declared in its enum declaration.
<code>int</code>	<code>ordinal()</code> Returns the ordinal of this enumeration constant (its position in its enum declaration, where the initial constant is assigned an ordinal of zero).
<code>String</code>	<code>toString()</code> Returns the name of this enum constant, as contained in the declaration.
<code>static E</code>	<code>valueOf(String name)</code> Returns the enum constant of the specified name.
<code>static E[]</code>	<code>values()</code> Returns an array containing all possible values defined by the enum.

Classe LocalDate

<code>boolean</code>	<code>isAfter(ChronoLocalDate other)</code> Checks if this date is after the specified date.
<code>boolean</code>	<code>isBefore(ChronoLocalDate other)</code> Checks if this date is before the specified date.
<code>static LocalDate</code>	<code>now()</code> Obtains the current date from the system clock in the default time-zone.
<code>static LocalDate</code>	<code>of(int year, int month, int dayOfMonth)</code> Obtains an instance of <code>LocalDate</code> from a year, month and day.
<code>LocalDate</code>	<code>plusDays(long daysToAdd)</code> Returns a copy of this <code>LocalDate</code> with the specified number of days added.
<code>long</code>	<code>until(LocalDate endExclusive, TemporalUnit unit)</code> Calculates the amount of time until another date in terms of the specified unit (e.g., <code>ChronoUnit.YEARS</code>).

Interface Comparator<T>

<code>int</code>	<code>compare(T o1, T o2)</code> Compares its two arguments for order. Returns a negative integer, zero, or a positive integer as the first argument is less than, equal to, or greater than the second.
------------------	---

Interface Iterable<T>

<code>Iterator<T></code>	<code>iterator()</code> Returns an iterator over elements of type <code>T</code> .
--------------------------------	---

Interface Comparable<T>

<code>int</code>	<code>compareTo(T o)</code> Compares this object with the specified object for order. Returns a negative integer, zero, or a positive integer as this object is less than, equal to, or greater than the specified object.
------------------	---